

User Manual

for S32 OCOTP Driver

Document Number: UM11OCOTPASR4.4 Rev0000R4.0.2 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	7
3.4 Deviations from Requirements	8
3.5 Driver Limitations	8
3.5.1 For Driver	8
3.5.2 For Errata	8
3.6 Driver usage and configuration tips	8
3.6.1 Fuse Map Table	9
3.6.2 Shadow register reload	11
3.6.3 Fuse protection and Shadow register overrides protection (Locking Feature).	11
3.6.4 ECC Protection	13
3.6.5 CRC Test	14
3.6.6 Ocotp Timeout Supervision	15
3.6.7 Configuration	16
3.7 Runtime errors	18
3.8 Symbolic Names Disclaimer	19
4 Tresos Configuration Plug-in	20
4.1 Module Ocotp	21
4.2 Container OcotpGeneral	22
4.3 Parameter OcotpDevErrorDetect	22
4.4 Parameter OcotpIpDevErrorDetect	22
4.5 Parameter OcotpEnableUserModeSupport	24
4.6 Parameter OcotpEnableMultiCore	24
4.7 Parameter OcotpWriteEFuseApi	25
4.8 Parameter OcotpWriteShadowRegisterApi	25
4.9 Parameter OcotpReadEFuseApi	26
4.10 Parameter OcotpReadShadowRegisterApi	26
4.11 Parameter OcotpReadApplicationInformationApi	27
4.12 Parameter OcotpWriteApplicationInformationApi	27

4.13 Parameter OcotpReadStickyInformationApi	28
4.14 Parameter OcotpWriteStickyInformationApi	28
4.15 Parameter OcotpEccErrorInjectionApi	29
4.16 Parameter OcotpEccGetErrorApi	29
4.17 Parameter OcotpSetModeApi	30
4.18 Parameter OcotpCrcTestApi	30
4.19 Parameter OcotpGetStatusApi	31
4.20 Parameter OcotpTimeoutMethod	31
4.21 Parameter OcotpWriteTimeout	32
4.22 Parameter OcotpReadTimeout	32
4.23 Parameter OcotpSetModeTimeout	33
4.24 Parameter OcotpCrcValidateTimeout	33
4.25 Container OcotpConfigList	34
4.26 Container OcotpHwUnit	34
4.27 Parameter OcotpHardwareIndex	35
4.28 Parameter OcotpInstance	35
4.29 Parameter OcotpSecureModeEnable	36
4.30 Reference OcotpEcucPartitionRef	36
4.31 Container OcotpChannelList	37
4.32 Container OcotpChannel	37
4.33 Parameter OcotpChannelNumber	38
4.34 Parameter OcotpWord	38
4.35 Parameter OcotpEccProtectionEnabled	39
4.36 Parameter OcotpReadProtectionEnabled	39
4.37 Parameter OcotpWriteProtectionEnabled	40
4.38 Parameter OcotpWriteOperationBlockingEnabled	40
4.39 Parameter OcotpCrcEnable	41
4.40 Parameter OcotpCRCTestRegionStart	41
4.41 Parameter OcotpCRCTestRegionEnd	42
4.42 Parameter OcotpCRCTestFuseWord	43
4.43 Parameter OcotpGPRRegisterIndex	43
4.44 Parameter OcotpSTRRegisterIndex	44
4.45 Reference OcotpHwRef	44
4.46 Container CommonPublishedInformation	45
4.47 Parameter ArReleaseMajorVersion	45
4.48 Parameter ArReleaseMinorVersion	46
4.49 Parameter ArReleaseRevisionVersion	46
4.50 Parameter ModuleId	47
4.51 Parameter SwMajorVersion	47
4.52 Parameter SwMinorVersion	48

4.53 Parameter SwPatchVersion	48
4.54 Parameter VendorApiInfix	49
4.55 Parameter VendorId	49
5 Module Index	51
5.1 Software Specification	51
6 Module Documentation	52
6.1 OCOTP	52
6.1.1 Detailed Description	52
6.1.2 Data Structure Documentation	54
6.1.3 Macro Definition Documentation	55
6.1.4 Types Reference	62
6.1.5 Enum Reference	62
6.1.6 Function Reference	63
6.2 OCOTP_IP	73
6.2.1 Detailed Description	73
6.2.2 Data Structure Documentation	74
6.2.3 Types Reference	74
6.2.4 Enum Reference	75
6.2.5 Function Reference	76



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes for On-Chip One Time Programmable (OCOTP) Driver for S32. OCOTP driver configuration parameters and deviations from the specification are described in Driver chapter of this document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525
- s32r45_bga780

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
AUTOSAR	Automotive Open System Architecture
DET	Development Error Tracer
ECU	Electronic Control Unit
MCU	Micro Controller Unit
OS	Operating System
MSB	Most Significant Bit
N/A	Not Applicable
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language
OCOTP	On-Chip One Time Programmable
ECC	Error Correcting Code
CRC	Cyclic Redundancy Check

2.5 Reference List

#	Title	Version
1	S32G2 Reference Manual	Rev. 7, February 2023
2	S32G3 Reference Manual	Rev.3, Feb 2023
3	S32R45 Reference Manual	Rev. 3, 12/2021
4	S32G2: Mask Set Errata for Mask 0P77B	v3.0
4	S32G2: Mask Set Errata for Mask 1P77B	v1.0
5	S32G3: Mask Set Errata for Mask 1P72B	v2.1
6	S32R45: Mask Set Errata for Mask P57D	Rev. 2.0
7	S32G2 Data Sheet	Rev 6, Dec 2022
8	S32G3 Data Sheet	Rev 2, RC, Feb 2023
9	VR5510 Data Sheet	Rev 5, April 2022
10	S32R45 Data Sheet	Rev. 3, RC — 11/2022



Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)

3.1 Requirements

Deviations from requirements are described in [Deviations from Requirements](#) chapter of this document.

3.2 Driver Design Summary

The OCOTP (On-Chip One Time Programmable) driver provides interface to program for Fusebox. You can use these fuses for several purposes, including:

- Unique chip identifiers
- Mask revision numbers
- Cryptographic keys
- Boot configurations
- Various control signals requiring permanent non-volatility

Fusebox include efuses, in divided in 16 bank, each bank is 8 words, each word efuses is 32 eFuses. Each eFuse bit cannot be reset to 0 if set to 1.

Ocotp driver is designed to provide APIs to implement the following functions:

- Programming and reading of 32-bit eFuse word via the register interface
- ECC-corrected read of shadow registers
- Override of shadow register's value without programming of eFuses
- Calculation of CRC (Polynomial CRC-32/MPEG-2) on eFuse words
- 32-bit read and write accesses to eFuse and shadow registers.

The user performs programming with the eFuse Word and corresponding Shadow registers by configuring the channels. Each channel includes:

- Channel name, channel index that is used to call parameters for the api in the ocotp driver
- An eFuse word, includes information:
 - the eFuse word address and corresponding Shadow register address
 - the ECC protect feature for this eFuse word is enabled or disabled
- The CRC test feature is enabled or disabled (not related with eFuse word above). If enable, this field will contain the information:
 - "Start eFuse word" address and "End eFuse word" address of region efuse words that need to be checked for crc.
 - CRC eFuse word address which will store the crc value calculated by the software
- Write Operation Blocking enabled or disable.
- A General purpose registers.

Index	Name	Ocotp Cha...	Ocotp HW Reference	eFuse Word	Ecc Prote...	Read P...	Write ...	Write Op...	Ocotp Cr...	CRC Test ...	CRC Test ...	CRC Test ...
0	OcotpChannel_0	0	/Ocotp/Ocotp/OcotpConf...	OCOTP_GP1	☒	☐	☐	☒	☐	OCOTP_GP1	OCOTP_GP...	OCOTP_CR...
1	OcotpChannel_1	1	/Ocotp/Ocotp/OcotpConf...	OCOTP_GP2	☐	☒	☐	☒	☐	OCOTP_GP...	OCOTP_U...	OCOTP_CR...
2	OcotpChannel_2	2	/Ocotp/Ocotp/OcotpConf...	OCOTP_CRC2	☐	☐	☐	☒	☒	OCOTP_GP1	OCOTP_GP2	OCOTP_CR...
3	OcotpChannel_3	3	/Ocotp/Ocotp/OcotpConf...	OCOTP_GP5_127_96	☒	☐	☐	☒	☐	OCOTP_U...	OCOTP_BA...	OCOTP_CR...

Figure 3.1 Channel configuration

3.3 Hardware Resources

OCOTP Hardware Resource includes 128 eFuseWords (called eFuse Box). However, the OCOTP driver currently only supports eFuse programming from the FuseMap table. For more details, please see the "S32_Fuse_Map_Tables" excel file attached in Reference Manual document.

3.4 Deviations from Requirements

The driver deviates from the OCOTP Driver software specification in some places. There are also some additional requirements (on top of requirements detailed in OCOTP Driver software specification) which need to be satisfied for correct operation.

- Deviations Status Column Description

Term	Definition
N/S	Not In Scope
N/F	Not Fully Implemented
N/I	Not Implemented

None.

3.5 Driver Limitations

3.5.1 For Driver

- S32XX does not support FBXC.

3.5.2 For Errata

- S32G2XX:
 - ERR051655 PFLASH: ERR051655: OCOTP: Incorrect data may be read from fuses after software power down and up of the fusebox
 - * There is no software workaround for this Errata. A destructive reset or a standby exit must be issued before Ocotp_SetMode, Ocotp_ReadEFuse or Ocotp_WriteEFuse, if the Mode is set to OCOTP←_POWERDOWN_MODE.

3.6 Driver usage and configuration tips

- [Fuse Map Table](#)
- [Shadow register reload](#)
- [Fuse protection and Shadow register overrides protection \(Locking Feature\).](#)
- [ECC Protection](#)
- [CRC Test](#)
- [Ocotp Timeout Supervision](#)
- [Configuration](#)

3.6.1 Fuse Map Table

Fuse map is described in detail in the excel file attached to the Reference Manual.

	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1														
2		Fuse Bank :	Fuse Bank : Word	Lock Bit Name	Address (Hex)	31	30	29	28	27	26	25	24	23
3	0000	0002	LOCK_CUSTOMER_LOCK_BITS	208	LOCK_CUSTOMER_LOCK_BITS									NXP Reserved
4	0000	0003	LOCK_CUSTOMER_LOCK_BITS	20C										
5	0000	0004	TESTER_LOCK	210	UNIQUE_ID[31:0]									
6	0000	0005		214	UNIQUE_ID[63:32]									
7	0000	0006	TESTER_LOCK	218	NXP Reserved									
8	0001	0005	BOOT_CFG_LOCK	234										
9	0001	0006	BOOT_CFG_LOCK	238										
10	0001	0007	BOOT_CFG_LOCK	23C										
11	0002	0000	MISC_CONF_LOCK	240										
12	0002	0001	MISC_CONF_LOCK	244										
13	0002	0002	MAC_ADDR_LOCK	248										
14	0002	0003	MAC_ADDR_LOCK	24C										
15	0002	0004	MAC_ADDR_LOCK	250										
16	0004	0006	ANALOG_LOCK	298										CFG_DAC_TRIM_3_VALID
17	0005	0001	GP1_LOCK	2A4										
18	0005	0002	GP2_LOCK	2A8										
19	0006	0007		2DC	CRC0									
20	0007	0000		2E0	CRC1									
21	0007	0001		2E4	CRC2									
22	0011	0000	GP5_LOCK	360	GP5[31:0]									
23	0011	0001	GP5_LOCK	364	GP5[63:32]									

Figure 3.2 Fuse map table

FuseMap is divided into 16 Bank, each Bank contains 8 Word.

S.No.	Fuse Bank	Fuse Bank : Word
0008	0000	0002
0012	0000	0003
0016	0000	0004
0020	0000	0005
0024	0000	0006
0052	0001	0005
0056	0001	0006
0060	0001	0007

Figure 3.3 eFuse Words

Each eFuse Word has different functions. Fuse Map is described in the "Fuse Description" sheet.

For example:

-Words used to contain the MAC address:

13	0072	0002	0002	MAC_ADDR_LOCK	248	MAC0_ADDR[31:24]	
14	0076	0002	0003	MAC_ADDR_LOCK	24C	NXP Reserved	

Figure 3.4 eFuse Words used to contain the MAC address

Driver

-Words used to configure BootROM

8	0052	0001	0005	BOOT_CFG_LOCK	234	BOOT_CFG1[31:24]	
9	0056	0001	0006	BOOT_CFG_LOCK	238	BOOT_CFG2[31:24]	
	0060	0001	0007	BOOT_CFG_LOCK	23C	BOOT_CFG3[31:24]	

Figure 3.5 eFuse Words used for boot configuration

-BootROM config is described in detail in the "Boot ROM Fuse Description" sheet

[illegible]

Figure 3.6 Boot ROM fuse description

-Word 2 and 3 of Bank 0 contain the eFuse bits used to enable the locking feature (read protection, write protection) of other eFuse. Details see in the "Fuse protection and overrides(Lock Feature)".

0008	0000	0002	LOCK_CUSTOMER_LOCK_BITS	208	LOCK_CUSTOMER_LOCK_BITS		NXP Reserved
0012	0000	0003	LOCK_CUSTOMER_LOCK_BITS	20C		NXP Reserved	
0016	0000	0004	LOCK_CUSTOMER_LOCK_BITS	210	UNIQUE_ID34-01		

Figure 3.7 eFuse Words used to enable the locking feature

-Bank6_Word7, Bank7_Word0, Bank7_Word1 (CRC0, CRC1, CRC2) is the place to write the CRC value calculated by the software (or user). If Ocotp is required to check the CRC for a certain area, the hardware will calculate and compare it with the value of 1 of these 3 eFuseWord to send the confirmation signal. For details see "CRC Test".

0220	0006	0007		2DC	CRC0
0224	0007	0000		2E0	CRC1
0228	0007	0001		2E4	CRC2

Figure 3.8 eFuse Words used for CRC test feature

-The eFuses in the figure below are the eFuses that contain the value of ECC protection for other eFusewords. See "ECC Protection" for details.

3.6.2 Shadow register reload

The Shadow Registers only reload the values of the corresponding eFuse words when the MCU is reset.

Caution that all the features like Lock Protect, ECC Protect, CRC Test when enabled will get value from Shadow registers, not on eFuse Box. So, the feature is only really active when the shadow register is reloaded. If user don't want to reset the MCU, user can update the Shadow Register value (if possible) by reading the corresponding eFuse Word and rewriting it on the Shadow Register.

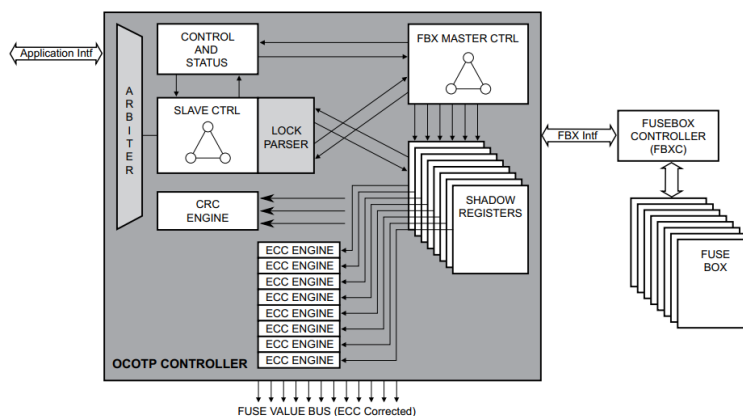


Figure 3.9 OCOTP system level diagram

3.6.3 Fuse protection and Shadow register overrides protection (Locking Feature).

Notes:

- Typically the first few words of the eFuses Box define the eFuse word locks.
- There can be more than one eFuse with the same lock.
- eFuse word can have any number of these locks, and can also have no locks as mentioned in the chip fuse map.
- eFuse/shadow protection is chip-specific implementation dependent.
- eFuse word locks can be protected by LOCK_CUSTOMER_LOCK_BITS of Bank0_Word2.

0008	0000	0002	LOCK_CUSTOMER_LOCK_BITS	208	LOCK_CUSTOMER_LOCK_BITS	NXP Reserved	NXP Reserved
0012	0000	0003	LOCK_CUSTOMER_LOCK_BITS	20C	NXP Reserved	NXP Reserved	NXP Reserved
0016	0000	0004	TESTED_LOCK	210	UNIQUE ID(31-0)	NXP Reserved	NXP Reserved

Figure 3.10 eFuse Words used to enable the locking feature

LockBit Format (Bits [1:0])	
x1	1x
RP (ReadProtect)	WP (WriteProtect) + OP (OverRideProtect)
Read to shadow register is locked Read to Fuse is locked.	Write to Fuse is Locked Write to shadow register is Locked Read is allowed

Figure 3.11 LockBit description

The Ocotp driver support user protect writing/reading eFuse words in the channel configuration. When the user enables the "Read Protection Enabled" node and uses the `Ocotp_WriteEFuse()` API to write to the eFuse word, Read Protection with this eFuse word will be enabled. Similar to the "Write Read Protection Enabled" node.

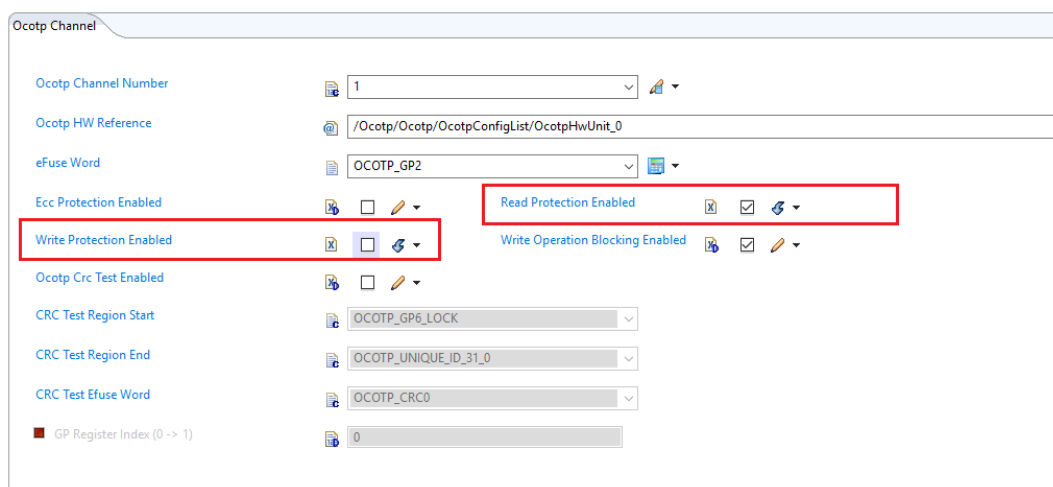


Figure 3.12 Locking Feature

Also, user can write into the bank0 word2 or word3 to directly enable Read Protect, Write Protect for many eFuse words.

Example:

- Assuming GP1 has been written and user want Read Protect, Write Protect this eFuse Word and Read Protect, Override protect the responsible shadow register. So, user need write 0x11b into 2 bit GP1_LOCK of Bank0←_Word2 eFuse Word.

	2	1	0	ECC Supported
GP2_LOCK		GP1_LOCK		Y
		GP6_LOCK		Y

Figure 3.13 LockBit position

More specifically, the user can follow the following steps to Read Protect, Write Protect:

- Read BANK0_WORD2 efuseWord get Value by `Ocotp_ReadEFuse()` API
- Addition 0x11b into GP1_LOCK of Bank0_Word2 : Value = Value | (0x11b << GP1_LOCK_POSITON)
- Write Value into BANK0_WORD2 efuseWord by `Ocotp_WriteEFuse()` API
- Reset MCU to update Shadow register. (If only protect read, you can write Value to Shadow register so you don't have to reset chip)

3.6.4 ECC Protection

ECC is a feature used to detect and correct errors of eFuse words data occurring in OCOTP memory. ECC makes eFuse Words immune to single bit errors: the data that is read from each eFuse word is always the same as the data that had been written to it, even if one of the eFuse bits actually stored has been flipped to the wrong state. In addition, OCOTP also supports double error detection.

Notes:

- ECC protection is applied directly to the value of the Shadow register
- Typically the last eFuse words of the eFuse box are used to hold the values to enable ECC for the eFuses Words.

21	0352	0011	0000	GP5_LOCK	360	GP5[31:0]	
22	0356	0011	0001	GP5_LOCK	364	GP5[63:32]	
23	0360	0011	0002	GP5_LOCK	368	GP5[95:64]	
24	0364	0011	0003	GP5_LOCK	36C	GP5[127:96]	
25	0368	0011	0004	GP5_LOCK	370	GP5[159:128]	
26	0372	0011	0005	GP5_LOCK	374	GP5[191:160]	
27	0376	0011	0006	GP6_LOCK	378	GP6[30:0]	
28	0380	0011	0007	GP6_LOCK	37C	GP6[62:31]	
29	0384	0012	0000	GP6_LOCK	380	GP6[94:63]	
30	0388	0012	0001	GP6_LOCK	384	GP6[126:95]	
31	0392	0012	0002	GP6_LOCK	388	GP6[158:127]	
32	0412	0012	0007		39C	ECC for bank 0, word 3	
33	0424	0013	0002		3A8	ECC for bank 1, word 7	
34	0428	0013	0003		3AC	ECC for bank 2, word 3	
35	0432	0013	0004		3B0	ECC for bank 2, word 7	
36	0452	0014	0001		3C4	ECC for bank 5, word 3	
37	0464	0014	0004		3D0	ECC for bank 6, word 7	
38	0468	0014	0005		3D4	ECC for bank 7, word 3	
39	0500	0015	0005		3F4	ECC for bank 11, word 3	
40	0504	0015	0006		3F8	ECC for bank 11, word 7	
41	0508	0015	0007		3FC	ECC for bank 12, word 3	
42							

Figure 3.14 ECC protection description

- The ECC for each eFuse Word is 8 bits. Where 1 bit is used to enable or disable ECC protection, 7 bits contain the ECC value that is computed in software based on the value that the user writes into the eFuse Word

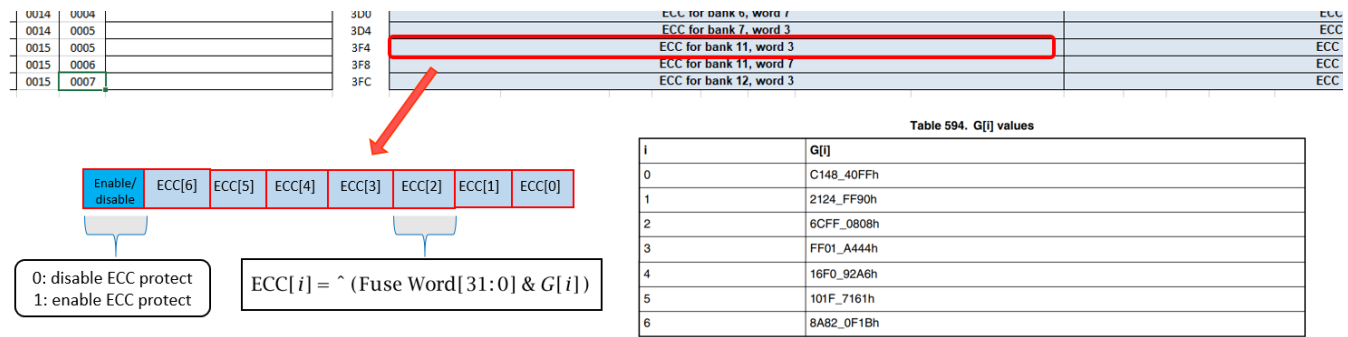


Figure 3.15 ECC calculation

- The Ocotp driver allows user to enable the ECC feature only by enabling ECC in channel config. When the user enables ECC and uses the [Ocotp_WriteEFuse\(\)](#) API to write to the eFuse word, Ecc will be enabled in this API.

Index	Name	Ocotp Cha...	Ocotp HW Reference	eFuse Word	Ecc Protection Enabled	Read Protection Enabled	Write Protection Enabled	Write Operation Blocking En...
0	OcotpChannel_0	0	/Ocotp/Ocotp/OcotpConfi...	OCOTP_GP1	☒	☒	☒	☒
1	OcotpChannel_1	1	/Ocotp/Ocotp/OcotpConfi...	OCOTP_GP2	☐	☒	☒	☒
2	OcotpChannel_2	2	/Ocotp/Ocotp/OcotpConfi...	OCOTP_CRC2	☐	☒	☒	☒
3	OcotpChannel_3	3	/Ocotp/Ocotp/OcotpConfi...	OCOTP_GP5_127_96	☒	☒	☒	☒

Figure 3.16 ECC protection enabled

3.6.5 CRC Test

Users can use the CRC checking feature to validate the content of an eFuse words region, including read protected eFuses. Some eFuse is reserved for storing the CRC value of the pre-defined eFuse portion.

0220	0006	0007	2DC	CRC0
0224	0007	0000	2E0	CRC1
0228	0007	0001	2E4	CRC2

Figure 3.17 eFuse Words used for CRC test feature

Ocotp driver provides the [Ocotp_CalculateAndWriteCrcValue\(\)](#) API to compute CRC32 for an array of data and write to a CRC eFuse Word. The [Ocotp_CrcValidate\(\)](#) API will compare the value in CRC eFuse Word with the value computed by the hardware.

Example: The user wrote Data in GP1 and GP2, and protected read for GP1. The user wants to verify the written value but because GP1 is read-locked, cannot use the [Ocotp_ReadShadowRegister\(\)](#) API or the [Ocotp_ReadEFuse\(\)](#) API to read GP1, so CRC check will be used.

15	MAC_ADDR_LOCK	250		
16	GP1_LOCK	2A4		
17	GP2_LOCK	2A8		
18		2DC	CRC0	
19		2E0	CRC1	
20		2E4	CRC2	
21	GP5_LOCK	360	GP5[31:0]	

Figure 3.18 CRC test feature description

Configure the channel to test CRC:

Ocotp Channel

Ocotp Channel Number: 2

Ocotp HW Reference: /Ocotp/Ocotp/OcotpConfigList/OcotpHwUnit_0

eFuse Word: OCOTP_GP2

Ecc Protection Enabled: ☐

Read Protection Enabled: ☐

Write Protection Enabled: ☐

Write Operation Blocking Enabled: ☒

Ocotp Crc Test Enabled: ☒

CRC Test Region Start: OCOTP_GP1

CRC Test Region End: OCOTP_GP2

CRC Test Efuse Word: OCOTP_CRC2

GP Register Index (0 -> 1): 0

Figure 3.19 Configuration for CRC test

Users can follow the following sequence to check the CRC:

- Call `Ocotp_CalculateAndWriteCrcValue()` API with the Data and Channel enabled CRC.
- Call `Ocotp_CrcValidate()` API to validate data.

Note: CRC Test feature is implemented by Ocotp on Shadow register, not directly on eFuse Words.

3.6.6 Ocotp Timeout Supervision

Ocotp will be in a busy state if it is in processes:

- Write to eFuse
- Read from eFuse
- CRC test

Therefore, some of the following APIs need to be configured Timeout for OCOTP to complete the process.

Driver

- [Ocotp_ReadEFuse\(\);](#)
- [Ocotp_WriteEFuse\(\);](#)
- [Ocotp_CalculateAndWriteCrcValue\(\);](#)
- [Ocotp_CrcValidate\(\);](#)
- [Ocotp_SetMode\(\);](#)

User can refer to the configuration for Timeout as shown below.

Ocotp Timeout Method	 OSIF_COUNTER_DUMMY  
Ocotp Write Timeout (0 -> 4294967295)	 100  
Ocotp Read Timeout (0 -> 4294967295)	 100  
Ocotp Set Mode Timeout (0 -> 4294967295)	 100  
Ocotp Crc Validate Timeout (0 -> 4294967295)	 100  

Figure 3.20 Configuration for timeout

Notes (Node "Write Operation Blocking Enabled"): When configure a channel, there is a node "Write Operation Blocking Enabled". This node will allow you to enable whether or not to blocking during eFuse writing. If enabled Blocking, writing to eFuse Words will be waited for busy bit is cleared to ensure Ocotp completes the process.

If disable Blocking, Ocotp_WriteEFuse function will be completed without waiting for busy bit is cleared.

Write Operation Blocking Enabled  ☒  

Figure 3.21 Write operation blocking enabled

APIs related to blocking:

- [Ocotp_WriteEFuse\(\);](#)
- [Ocotp_CalculateAndWriteCrcValue\(\);](#)

3.6.7 Configuration

A channel will contain some independent information:

- General information:
 - Ocotp Channel Number
 - Ocotp HW Reference
 - Write Operation Blocking Enabled : This applies to both write eFuse Word and CRC tests

- An eFuse word information:
 - eFuse Word : the eFuse Word address and corresponding Shadow Register address
 - Ecc Protection Enabled : the ECC protect feature for this eFuse word is enabled or disabled
- The CRC Test information: The CRC test feature is enabled or disabled (not related with eFuse word above).If enable, this field will contain the information:
 - CRC Test Region Start
 - CRC Test Region End : region which hardware will get value to CRC test
 - CRC eFuse Word : address which will store the crc value calculated by the software
- A General purpose registers information.
- A Sticky register information (only support on S32G3)

The screenshot displays the 'Ocotp Channel' configuration window. It features several sections with color-coded labels on the left:

- General Information (Blue):** Includes 'Ocotp Channel Number' (set to 0) and 'Ocotp HW Reference' (set to /Ocotp/Ocotp/OcotpConfigList/OcotpHwUnit_0).
- An Efuse word information (Red):** Includes 'eFuse Word' (set to OCOTP_GP6_127_96), 'Ecc Protection Enabled' (checkbox), 'Write Protection Enabled' (checkbox), and 'Write Operation Blocking Enabled' (checkbox).
- CRC Test information (Green):** Includes 'Ocotp CRC32 Test Enable*' (checkbox), 'CRC Test Region Start' (set to OCOTP_GP1), 'CRC Test Region End' (set to OCOTP_GP6_LOCK), and 'CRC Test Efuse Word' (set to OCOTP_CRC0).
- A general purpose information (Yellow):** Includes 'GP Register Index (0 -> 1)' (set to 0).
- A sticky register information (Light Blue):** Includes 'STR Register Index (0 -> 1)' (set to 0).

Figure 3.22 Channel configuration information

Note about Sticky configuration:

- Sticky registers only support on S32G3 derivative. These are general-purpose registers. You can write a 1 to a bit in the fields of these registers only one time per reset
- With derivatives not be supported, value of sticky nodes must be "false" at General:

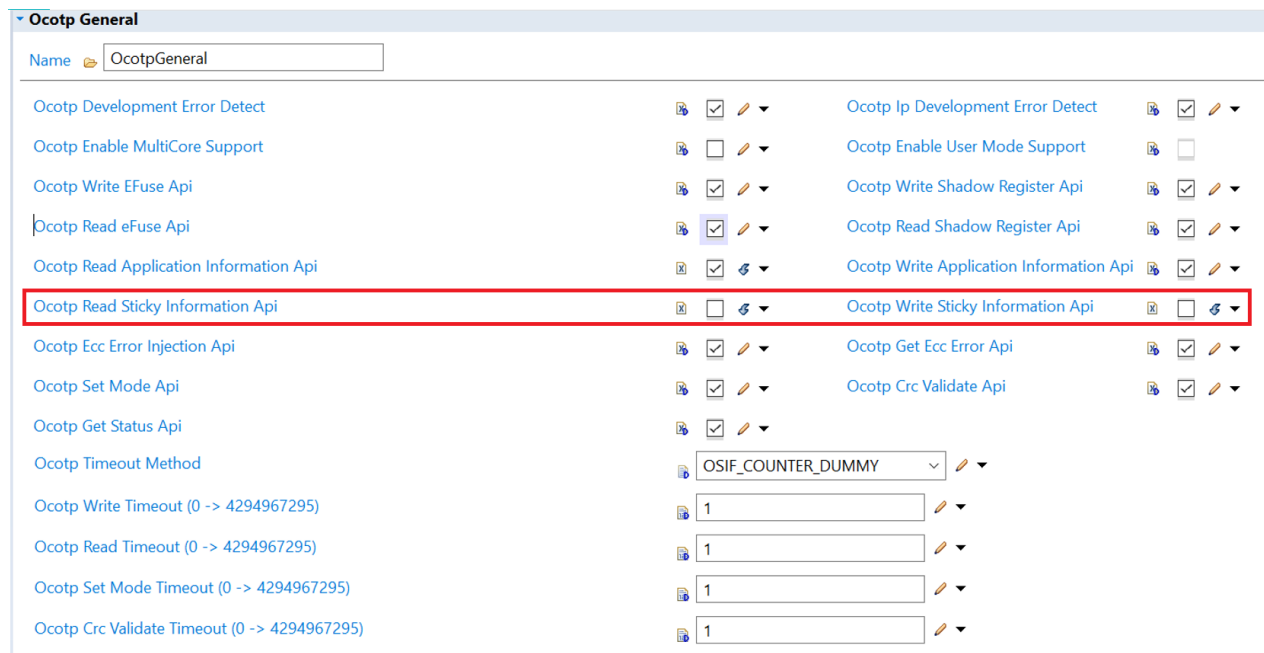


Figure 3.23 Sticky nodes on General

3.7 Runtime errors

- The driver supports runtime generation of the errors listed in the Table:

Function	Error code	Condition triggering the error
Ocotp_WriteEFuse()	OCOTP_E_TIMEOUT	(Ocotp Timeout Supervision Enabled == ON) and Ocotp is still busy when the timeout is over.
Ocotp_ReadEFuse()	OCOTP_E_TIMEOUT	(Ocotp Timeout Supervision Enabled == ON) and Ocotp is still busy when the timeout is over.
Ocotp_SetMode()	OCOTP_E_TIMEOUT	(Ocotp Timeout Supervision Enabled == ON) and Ocotp is still busy when the timeout is over.
Ocotp_CalculateAndWriteCrcValue()	OCOTP_E_TIMEOUT	(Ocotp Timeout Supervision Enabled == ON) and Ocotp is still busy when the timeout is over.
Ocotp_CrcValidate()	OCOTP_E_TIMEOUT	(Ocotp Timeout Supervision Enabled == ON) and Ocotp is still busy when the timeout is over.

- The driver supports Transient faults generation of the errors listed in the Table.

Function	Error Code	Condition triggering the error
Ocotp_WriteEFuse()	OCOTP_E_WRITE_FAILED	Write operation failed.

Function	Error Code	Condition triggering the error
Ocotp_WriteShadowRegister()	OCOTP_E_WRITE_FAILED	Write operation failed.(in the case of writing to a write-protected Shadow register).
Ocotp_CalculateAndWriteCrcValue()	OCOTP_E_WRITE_FAILED	Write operation failed.
Ocotp_ReadEFuse()	OCOTP_E_READ_FAILED	Read operation failed.
Ocotp_ReadShadowRegister()	OCOTP_E_READ_FAILED	Read operation failed.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like:

```
#define <Mip>Conf_<Container_ShortName>_<Container_ID>
```

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module [Ocotp](#)
 - Container [OcotpGeneral](#)
 - * Parameter [OcotpDevErrorDetect](#)
 - * Parameter [OcotpIpDevErrorDetect](#)
 - * Parameter [OcotpEnableUserModeSupport](#)
 - * Parameter [OcotpEnableMultiCore](#)
 - * Parameter [OcotpWriteEFuseApi](#)
 - * Parameter [OcotpWriteShadowRegisterApi](#)
 - * Parameter [OcotpReadEFuseApi](#)
 - * Parameter [OcotpReadShadowRegisterApi](#)
 - * Parameter [OcotpReadApplicationInformationApi](#)
 - * Parameter [OcotpWriteApplicationInformationApi](#)
 - * Parameter [OcotpReadStickyInformationApi](#)
 - * Parameter [OcotpWriteStickyInformationApi](#)
 - * Parameter [OcotpEccErrorInjectionApi](#)
 - * Parameter [OcotpEccGetErrorApi](#)
 - * Parameter [OcotpSetModeApi](#)
 - * Parameter [OcotpCrcTestApi](#)
 - * Parameter [OcotpGetStatusApi](#)
 - * Parameter [OcotpTimeoutMethod](#)
 - * Parameter [OcotpWriteTimeout](#)
 - * Parameter [OcotpReadTimeout](#)
 - * Parameter [OcotpSetModeTimeout](#)
 - * Parameter [OcotpCrcValidateTimeout](#)
 - Container [OcotpConfigList](#)
 - * Container [OcotpHwUnit](#)
 - Parameter [OcotpHardwareIndex](#)
 - Parameter [OcotpInstance](#)
 - Parameter [OcotpSecureModeEnable](#)
 - Reference [OcotpEcucPartitionRef](#)

- Container [OcotpChannelList](#)
 - * Container [OcotpChannel](#)
 - Parameter [OcotpChannelNumber](#)
 - Parameter [OcotpWord](#)
 - Parameter [OcotpEccProtectionEnabled](#)
 - Parameter [OcotpReadProtectionEnabled](#)
 - Parameter [OcotpWriteProtectionEnabled](#)
 - Parameter [OcotpWriteOperationBlockingEnabled](#)
 - Parameter [OcotpCrcEnable](#)
 - Parameter [OcotpCRCTestRegionStart](#)
 - Parameter [OcotpCRCTestRegionEnd](#)
 - Parameter [OcotpCRCTestFuseWord](#)
 - Parameter [OcotpGPRRegisterIndex](#)
 - Parameter [OcotpSTRRegisterIndex](#)
 - Reference [OcotpHwRef](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)

4.1 Module Ocotp

Configuration of the Ocotp module.

Included containers:

- [OcotpGeneral](#)
- [OcotpConfigList](#)
- [OcotpChannelList](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.2 Container OcotpGeneral

Container for general parameters of the ocotp driver. These parameters are always pre-compile.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.3 Parameter OcotpDevErrorDetect

Pre-processor switch to enable and disable development error detection.

true: Development error detection enabled.

false: Development error detection disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.4 Parameter OcotpIpDevErrorDetect

Pre-processor switch to enable and disable development error detection for IP layer.

true: Ocotp error detect for IP layer is enabled.

false: Ocotp error detect for IP layer is disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.5 Parameter OcotpEnableUserModeSupport

Ocotp module can run in user mode without any specific measures. The parameter is not used in the Ocotp implementation.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.6 Parameter OcotpEnableMultiCore

When this parameter is enabled, the OCOTP module will adapt to run with Multicore.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.7 Parameter OcotpWriteEFuseApi

Compile switch to enable and disable the Ocotp_WriteEFuse function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.8 Parameter OcotpWriteShadowRegisterApi

Compile switch to enable and disable the Ocotp_WriteShadowRegister function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.9 Parameter OcotpReadEFuseApi

Compile switch to enable and disable the Ocotp_ReadEFuse function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.10 Parameter OcotpReadShadowRegisterApi

Compile switch to enable and disable the Ocotp_ReadShadowRegister function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.11 Parameter OcotpReadApplicationInformationApi

Compile switch to enable and disable the Ocotp_ReadApplicationInformation function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.12 Parameter OcotpWriteApplicationInformationApi

Compile switch to enable and disable the Ocotp_WriteApplicationInformation function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.13 Parameter OcotpReadStickyInformationApi

Compile switch to enable and disable the Ocotp_ReadStickyInformation function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.14 Parameter OcotpWriteStickyInformationApi

Compile switch to enable and disable the Ocotp_WriteStickyInformation function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.15 Parameter OcotpEccErrorInjectionApi

Compile switch to enable and disable the Ocotp_EccErrorInject function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.16 Parameter OcotpEccGetErrorApi

Compile switch to enable and disable the Ocotp_EccGetError function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.17 Parameter OcotpSetModeApi

Compile switch to enable and disable the Ocotp_SetMode function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.18 Parameter OcotpCrcTestApi

Compile switch to enable and disable the Ocotp_CalculateAndWriteCrcValue and Ocotp_CrcValidate function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.19 Parameter OcotpGetStatusApi

Compile switch to enable and disable the Ocotp_GetStatus function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.20 Parameter OcotpTimeoutMethod

OcotpTimeoutMethod

Configures the timeout method.

Based on this selection a certain timeout method from OsIf will be used in the driver.

Tresos Configuration Plug-in

Note: If OSIF_COUNTER_SYSTEM or OSIF_COUNTER_CUSTOM are selected make sure the corresponding timer is enabled in OsIf General configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM']

4.21 Parameter OcotpWriteTimeout

Timeout value for write eFuse operation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	4294967295
min	0

4.22 Parameter OcotpReadTimeout

Timeout value for read eFuse operation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	4294967295
min	0

4.23 Parameter OcotpSetModeTimeout

Timeout value for set mode operation. See the chip data sheet for the required setup time (between powerdown and fusebox access).

We must respect this constraint. No accesses are possible to FBXC and eFuse during this time.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	4294967295
min	0

4.24 Parameter OcotpCrcValidateTimeout

Timeout value for CRC validate operation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	4294967295
min	0

4.25 Container OcotpConfigList

Container for runtime configuration parameters of the Ocotp driver.

Implementation Type: Ocotp_ConfigType.

Included subcontainers:

- [OcotpHwUnit](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.26 Container OcotpHwUnit

Ocotp Hardware Unit.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.27 Parameter OcotpHardwareIndex

Index of current hardware unit.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	255
min	0

4.28 Parameter OcotpInstance

Instance of this word channel

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	0
min	0

4.29 Parameter OcotpSecureModeEnable

Enable Secure Mode.

True: Secure mode is enabled.

Flase: Secure mode is disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.30 Reference OcotpEcucPartitionRef

Maps the OCOTP driver to zero or multiple ECUC partitions to make the driver API available in the according partition.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.31 Container OcotpChannelList

List of Otp channel.

Included subcontainers:

- [OcotpChannel](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.32 Container OcotpChannel

Ocotp Channel Configuration.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.33 Parameter OcotpChannelNumber

Index of current channel.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.34 Parameter OcotpWord

Ocotp Fuse Word.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OCOTP_BANK0_WORD2

Property	Value
literals	['OCOTP_BANK0_WORD2', 'OCOTP_GP6_LOCK', 'OCOTP_UNIQUE_ID_31_0', 'OCOTP_UNIQUE_ID_63_32', 'OCOTP_BANK0_WORD6', 'OCOTP_BOOT_CFG1', 'OCOTP_BOOT_CFG2', 'OCOTP_BOOT_CFG3', 'OCOTP_BANK2_WORD0', 'OCOTP_BANK2_WORD1', 'OCOTP_MAC0_ADDR_31_0', 'OCOTP_MAC0_ADDR_47_32', 'OCOTP_BANK2_WORD4', 'OCOTP_CFG_DAC_TRIM', 'OCOTP_GP1', 'OCOTP_GP2', 'OCOTP_CRC0', 'OCOTP_CRC1', 'OCOTP_CRC2', 'OCOTP_GP5_31_0', 'OCOTP_GP5_63_32', 'OCOTP_GP5_95_64', 'OCOTP_GP5_127_96', 'OCOTP_GP5_159_128', 'OCOTP_GP5_191_160', 'OCOTP_GP6_31_0', 'OCOTP_GP6_63_32', 'OCOTP_GP6_95_64', 'OCOTP_GP6_127_96', 'OCOTP_GP6_159_128', 'OCOTP_ECC_FOR_BANK0_WORD_3_0', 'OCOTP_ECC_FOR_BANK1_WORD_7_4', 'OCOTP_ECC_FOR_BANK2_WORD_3_0', 'OCOTP_ECC_FOR_BANK2_WORD_7_4', 'OCOTP_ECC_FOR_BANK5_WORD_3_0', 'OCOTP_ECC_FOR_BANK6_WORD_7_4', 'OCOTP_ECC_FOR_BANK7_WORD_3_0', 'OCOTP_ECC_FOR_BANK11_WORD_3_0', 'OCOTP_ECC_FOR_BANK11_WORD_7_4', 'OCOTP_ECC_FOR_BANK12_WORD_3_0']

4.35 Parameter OcotpEccProtectionEnabled

Enable Ecc protection for current fuse word.

TRUE: Ecc protection will be enable for current fuse after writing operation is successful.

FALSE: Ecc protection is disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.36 Parameter OcotpReadProtectionEnabled

Enable Read Protection for current fuse word.

TRUE: Read Protection will be enable for current fuse after writing operation is successful.

FALSE: Read Protection is disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.37 Parameter OcotpWriteProtectionEnabled

Enable Write Protection for current fuse word.

TRUE: Write Protection will be enable for current fuse after writing operation is successful.

FALSE: Write Protection is disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.38 Parameter OcotpWriteOperationBlockingEnabled

TRUE: function WriteEFuse is blocking.

FALSE: function WriteEFuse is NOT blocking.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.39 Parameter OcotpCrcEnable

Enable Crc test for current channel.

true: Crc test feature is enabled for current channel.

false: Crc test feature is disabled for current channel.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.40 Parameter OcotpCRCTestRegionStart

Start address of CRC test region

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OCOTP_BANK0_WORD2
literals	['OCOTP_BANK0_WORD2', 'OCOTP_GP6_LOCK', 'OCOTP_UNIQUE_ID_31_0', 'OCOTP_UNIQUE_ID_63_32', 'OCOTP_BANK0_WORD6', 'OCOTP_BOOT_CFG1', 'OCOTP_BOOT_CFG2', 'OCOTP_BOOT_CFG3', 'OCOTP_BANK2_WORD0', 'OCOTP_BANK2_WORD1', 'OCOTP_MAC0_ADDR_31_0', 'OCOTP_MAC0_ADDR_47_32', 'OCOTP_BANK2_WORD4', 'OCOTP_CFG_DAC_TRIM', 'OCOTP_GP1', 'OCOTP_GP2', 'OCOTP_CRC0', 'OCOTP_CRC1', 'OCOTP_CRC2', 'OCOTP_GP5_31_0', 'OCOTP_GP5_63_32', 'OCOTP_GP5_95_64', 'OCOTP_GP5_127_96', 'OCOTP_GP5_159_128', 'OCOTP_GP5_191_160', 'OCOTP_GP6_31_0', 'OCOTP_GP6_63_32', 'OCOTP_GP6_95_64', 'OCOTP_GP6_127_96', 'OCOTP_GP6_159_128', 'OCOTP_ECC_FOR_BANK0_WORD_3_0', 'OCOTP_ECC_FOR_BANK1_WORD_7_4', 'OCOTP_ECC_FOR_BANK2_WORD_3_0', 'OCOTP_ECC_FOR_BANK2_WORD_7_4', 'OCOTP_ECC_FOR_BANK5_WORD_3_0', 'OCOTP_ECC_FOR_BANK6_WORD_7_4', 'OCOTP_ECC_FOR_BANK7_WORD_3_0', 'OCOTP_ECC_FOR_BANK11_WORD_3_0', 'OCOTP_ECC_FOR_BANK11_WORD_7_4', 'OCOTP_ECC_FOR_BANK12_WORD_3_0']

4.41 Parameter OcotpCRCTestRegionEnd

Start address of CRC test region

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OCOTP_GP6_LOCK

Property	Value
literals	['OCOTP_BANK0_WORD2', 'OCOTP_GP6_LOCK', 'OCOTP_UNIQUE_ID_31_0', 'OCOTP_UNIQUE_ID_63_32', 'OCOTP_BANK0_WORD6', 'OCOTP_BOOT_CFG1', 'OCOTP_BOOT_CFG2', 'OCOTP_BOOT_CFG3', 'OCOTP_BANK2_WORD0', 'OCOTP_BANK2_WORD1', 'OCOTP_MAC0_ADDR_31_0', 'OCOTP_MAC0_ADDR_47_32', 'OCOTP_BANK2_WORD4', 'OCOTP_CFG_DAC_TRIM', 'OCOTP_GP1', 'OCOTP_GP2', 'OCOTP_CRC0', 'OCOTP_CRC1', 'OCOTP_CRC2', 'OCOTP_GP5_31_0', 'OCOTP_GP5_63_32', 'OCOTP_GP5_95_64', 'OCOTP_GP5_127_96', 'OCOTP_GP5_159_128', 'OCOTP_GP5_191_160', 'OCOTP_GP6_31_0', 'OCOTP_GP6_63_32', 'OCOTP_GP6_95_64', 'OCOTP_GP6_127_96', 'OCOTP_GP6_159_128', 'OCOTP_ECC_FOR_BANK0_WORD_3_0', 'OCOTP_ECC_FOR_BANK1_WORD_7_4', 'OCOTP_ECC_FOR_BANK2_WORD_3_0', 'OCOTP_ECC_FOR_BANK2_WORD_7_4', 'OCOTP_ECC_FOR_BANK5_WORD_3_0', 'OCOTP_ECC_FOR_BANK6_WORD_7_4', 'OCOTP_ECC_FOR_BANK7_WORD_3_0', 'OCOTP_ECC_FOR_BANK11_WORD_3_0', 'OCOTP_ECC_FOR_BANK11_WORD_7_4', 'OCOTP_ECC_FOR_BANK12_WORD_3_0']

4.42 Parameter OcotpCRCTestFuseWord

CRC eFuse Word which will be used to compare.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OCOTP_CRC0
literals	['OCOTP_CRC0', 'OCOTP_CRC1', 'OCOTP_CRC2']

4.43 Parameter OcotpGPRegisterIndex

Index of general purpose register.

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	1
min	0

4.44 Parameter OcotpSTRRegisterIndex

Index of sticky registers. To edit this field, enable the node OcotpReadStickyInformationApi or OcotpWriteStickyInformationApi

Property	Value
type	ECUC-INTEGGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	1
min	0

4.45 Reference OcotpHwRef

Select Ocotp Hardware Reference

Property	Value
type	ECUC-REFERENCE-DEF

Property	Value
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Ocotp/OcotpConfigList/OcotpHwUnit

4.46 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.47 Parameter ArReleaseMajorVersion

Vendor specific: Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.48 Parameter ArReleaseMinorVersion

Vendor specific: Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.49 Parameter ArReleaseRevisionVersion

Vendor specific: Patch version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.50 Parameter ModuleId

Vendor specific: Module ID of this module.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	255
max	255
min	255

4.51 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.52 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.53 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION

Property	Value
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	2
max	2
min	2

4.54 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>__>VendorId>__<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.55 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

OCOTP	52
OCOTP_IP	73

Chapter 6

Module Documentation

6.1 OCOTP

6.1.1 Detailed Description

Data Structures

- struct [Ocotp_ChannelConfigType](#)
Ocotp channel configuration type. [More...](#)
- struct [Ocotp_ConfigType](#)
Ocotp configuration type. [More...](#)
- struct [Ocotp_StatusType](#)
Ocotp status type. [More...](#)

Macros

- [#define OCOTP_MODULE_ID](#)
ID of module and instance.
- [#define OCOTP_INSTANCE_ID](#)
- [#define OCOTP_INVALID_INDEX](#)
Invalid value of index and address.
- [#define OCOTP_INVALID_ADDRESS](#)
- [#define OCOTP_E_PARAM_CONFIG](#)
Development error codes (passed to DET).
- [#define OCOTP_E_PARAM_LENGTH](#)
- [#define OCOTP_E_PARAM_POINTER](#)
- [#define OCOTP_E_PARAM_CHANNEL](#)
- [#define OCOTP_E_PARAM_INSTANCE](#)
- [#define OCOTP_E_PARAM_MODE](#)
- [#define OCOTP_E_UNINIT](#)
- [#define OCOTP_E_ALREADY_INITIALIZED](#)
- [#define OCOTP_E_BUSY](#)

- `#define OCOTP_E_TIMEOUT`
Runtime error codes (passed to DET).
- `#define OCOTP_E_WRITE_FAILED`
Transient Faults codes (passed to DET).
- `#define OCOTP_E_READ_FAILED`
- `#define OCOTP_INIT_ID`
All service IDs (passed to DET).
- `#define OCOTP_DEINIT_ID`
- `#define OCOTP_WRITE_EFUSE_ID`
- `#define OCOTP_WRITE_SHADOW_REGISTER_ID`
- `#define OCOTP_READ_SHADOW_REGISTER_ID`
- `#define OCOTP_READ_APPLICATION_INFORMATION_ID`
- `#define OCOTP_WRITE_APPLICATION_INFORMATION_ID`
- `#define OCOTP_ECC_ERROR_INJECT_ID`
- `#define OCOTP_ECC_GET_ERROR_ID`
- `#define OCOTP_SET_MODE_ID`
- `#define OCOTP_CALCULATE_AND_WRITE_ECC_VALUE_ID`
- `#define OCOTP_CRC_VALIDATE_ID`
- `#define OCOTP_READ_EFUSE_ID`
- `#define OCOTP_GET_STATUS_ID`

Types Reference

- `typedef uint32 Ocotp_AddressType`
Ocotp address type.
- `typedef uint8 Ocotp_ChannelType`
Ocotp channel type.

Enum Reference

- `enum Ocotp_EccErrorType`
Ocotp ECC error type.
- `enum Ocotp_ModeType`
Ocotp mode operation.
- `enum Ocotp_HardwareStatusType`
Status of Ocotp hardware.

Function Reference

- void [Ocotp_Init](#) (const [Ocotp_ConfigType](#) *const ConfigPtr)
The function initializes Ocotp module.
- void [Ocotp_Deinit](#) (void)
The function de-initializes Ocotp module.
- Std_ReturnType [Ocotp_WriteEFuse](#) ([Ocotp_ChannelType](#) eFuseChannel, uint32 data)
The function writes data to efuse word.
- Std_ReturnType [Ocotp_WriteShadowRegister](#) ([Ocotp_ChannelType](#) eFuseChannel, uint32 data)
The function writes data to shadow register.
- Std_ReturnType [Ocotp_ReadEFuse](#) ([Ocotp_ChannelType](#) eFuseChannel, uint32 *pData)
The function reads data from efuse word.
- Std_ReturnType [Ocotp_ReadShadowRegister](#) ([Ocotp_ChannelType](#) eFuseChannel, uint32 *pData)
The function writes data to shadow register.
- Std_ReturnType [Ocotp_ReadApplicationInformation](#) ([Ocotp_ChannelType](#) eFuseChannel, uint32 *pData)
The function reads data from GP register.
- Std_ReturnType [Ocotp_WriteApplicationInformation](#) ([Ocotp_ChannelType](#) eFuseChannel, uint32 data)
The function writes data to GP register.
- Std_ReturnType [Ocotp_EccErrorInject](#) ([Ocotp_ChannelType](#) eFuseChannel, uint8 bit)
The function injects ECC error in the specified eFuse.
- Std_ReturnType [Ocotp_EccGetError](#) ([Ocotp_ChannelType](#) eFuseChannel, [Ocotp_EccErrorType](#) *pEccValue)
The function gets ECC error of the specified eFuse.
- Std_ReturnType [Ocotp_SetMode](#) (uint8 HwIndex, [Ocotp_ModeType](#) mode)
The function sets operation mode for Ocotp module.
- Std_ReturnType [Ocotp_CalculateAndWriteCrcValue](#) ([Ocotp_ChannelType](#) eFuseChannel, const uint32 *pData, uint32 dataSize)
The function writes the CRC value to the CRC fuse word.
- Std_ReturnType [Ocotp_CrcValidate](#) ([Ocotp_ChannelType](#) eFuseChannel)
The function validates the CRC.
- Std_ReturnType [Ocotp_GetStatus](#) (uint8 HwIndex, [Ocotp_StatusType](#) *pStatus)
The function gets status of Ocotp module.

6.1.2 Data Structure Documentation

6.1.2.1 struct Ocotp_ChannelConfigType

Ocotp channel configuration type.

A structure which is used to contain the parameters of channel used. Such as address of eFuse. Index of shadow register.

Definition at line 169 of file Ocotp_Types.h.

Data Fields

Type	Name	Description
Ocotp_AddressType	Ocotp_EFuseAddress	Address of Efuse
uint32	Ocotp_ShadowIndex	Index of Shadow register
Ocotp_AddressType	Ocotp_LockFuseAddress	Address of Lock Efuse
Ocotp_AddressType	Ocotp_StartCRCAddress	Start eFuse address for CRC calculation
Ocotp_AddressType	Ocotp_EndCRCAddress	End eFuse address for CRC calculation
Ocotp_AddressType	Ocotp_CRCFuseAddress	CRC eFuse address used for comparison in CRC test
uint8	Ocotp_ChannelNumber	Channel number
uint8	Ocotp_GPRegisterIndex	Index of GP register
uint32	Ocotp_LockValue	Lock value
Ocotp_InstanceType	Ocotp_Instance	Instance Index
boolean	Ocotp_WriteBlocking	Write blocking
boolean	Ocotp_EccEnable	ECC Enable

6.1.2.2 struct Ocotp_ConfigType

Ocotp configuration type.

A structure which is used to contain the hardware configuration and the configuration of channels used.

Definition at line 194 of file Ocotp_Types.h.

Data Fields

Type	Name	Description
const Ocotp_Ip_ConfigType *	pHwConfig	User configuration structure
const Ocotp_ChannelConfigType (*	pChanelConfig[]	Ocotp channel configuration type

6.1.2.3 struct Ocotp_StatusType

Ocotp status type.

A structure which is used to contain the status of hardware (Busy or idle or errors occurred) and status of repair error flag(If FBXC exists).

Definition at line 209 of file Ocotp_Types.h.

6.1.3 Macro Definition Documentation

6.1.3.1 OCOTP_MODULE_ID

```
#define OCOTP_MODULE_ID
```

ID of module and instance.

Ocotp module ID

Definition at line 145 of file CDD_Ocotp.h.

6.1.3.2 OCOTP_INSTANCE_ID

```
#define OCOTP_INSTANCE_ID
```

Ocotp instance ID

Definition at line 146 of file CDD_Ocotp.h.

6.1.3.3 OCOTP_INVALID_INDEX

```
#define OCOTP_INVALID_INDEX
```

Invalid value of index and address.

Ocotp invalid index ID

Definition at line 151 of file CDD_Ocotp.h.

6.1.3.4 OCOTP_INVALID_ADDRESS

```
#define OCOTP_INVALID_ADDRESS
```

Ocotp invalid address ID

Definition at line 152 of file CDD_Ocotp.h.

6.1.3.5 OCOTP_E_PARAM_CONFIG

```
#define OCOTP_E_PARAM_CONFIG
```

Development error codes (passed to DET).

API service called with wrong parameter

Definition at line 158 of file CDD_Ocotp.h.

6.1.3.6 OCOTP_E_PARAM_LENGTH

```
#define OCOTP_E_PARAM_LENGTH
```

API service called with wrong parameter

Definition at line 159 of file CDD_Ocotp.h.

6.1.3.7 OCOTP_E_PARAM_POINTER

```
#define OCOTP_E_PARAM_POINTER
```

API service called with invalid pointer

Definition at line 160 of file CDD_Ocotp.h.

6.1.3.8 OCOTP_E_PARAM_CHANNEL

```
#define OCOTP_E_PARAM_CHANNEL
```

API service used with an invalid channel identifier or channel was not configured for the functionality of the calling API

Definition at line 161 of file CDD_Ocotp.h.

6.1.3.9 OCOTP_E_PARAM_INSTANCE

```
#define OCOTP_E_PARAM_INSTANCE
```

API service used with an invalid hardware unit identifier or hardware unit was not configured for the functionality of the calling API

Definition at line 162 of file CDD_Ocotp.h.

6.1.3.10 OCOTP_E_PARAM_MODE

```
#define OCOTP_E_PARAM_MODE
```

API service called with wrong parameter

Definition at line 163 of file CDD_Ocotp.h.

6.1.3.11 OCOTP_E_UNINIT

```
#define OCOTP_E_UNINIT
```

API service called without module initialization

Definition at line 164 of file CDD_Ocotp.h.

6.1.3.12 OCOTP_E_ALREADY_INITIALIZED

```
#define OCOTP_E_ALREADY_INITIALIZED
```

Ocotp_init service called module initialization

Definition at line 165 of file CDD_Ocotp.h.

6.1.3.13 OCOTP_E_BUSY

```
#define OCOTP_E_BUSY
```

API service called while driver still busy

Definition at line 166 of file CDD_Ocotp.h.

6.1.3.14 OCOTP_E_TIMEOUT

```
#define OCOTP_E_TIMEOUT
```

Runtime error codes (passed to DET).

Timeout exceeded

Definition at line 171 of file CDD_Ocotp.h.

6.1.3.15 OCOTP_E_WRITE_FAILED

```
#define OCOTP_E_WRITE_FAILED
```

Transient Faults codes (passed to DET).

Ocotp write failed (HW)

Definition at line 177 of file CDD_Ocotp.h.

6.1.3.16 OCOTP_E_READ_FAILED

```
#define OCOTP_E_READ_FAILED
```

Ocotp read failed (HW)

Definition at line 178 of file CDD_Ocotp.h.

6.1.3.17 OCOTP_INIT_ID

```
#define OCOTP_INIT_ID
```

All service IDs (passed to DET).

Service ID of Ocotp_Init function

Definition at line 190 of file CDD_Ocotp.h.

6.1.3.18 OCOTP_DEINIT_ID

```
#define OCOTP_DEINIT_ID
```

Service ID of Ocotp_Deinit function

Definition at line 191 of file CDD_Ocotp.h.

6.1.3.19 OCOTP_WRITE_EFUSE_ID

```
#define OCOTP_WRITE_EFUSE_ID
```

Service ID of Ocotp_WriteEfuse function

Definition at line 192 of file CDD_Ocotp.h.

6.1.3.20 OCOTP_WRITE_SHADOW_REGISTER_ID

```
#define OCOTP_WRITE_SHADOW_REGISTER_ID
```

Service ID of Ocotp_WriteShadow function

Definition at line 193 of file CDD_Ocotp.h.

6.1.3.21 OCOTP_READ_SHADOW_REGISTER_ID

```
#define OCOTP_READ_SHADOW_REGISTER_ID
```

Service ID of Ocotp_ReadShadow function

Definition at line 194 of file CDD_Ocotp.h.

6.1.3.22 OCOTP_READ_APPLICATION_INFORMATION_ID

```
#define OCOTP_READ_APPLICATION_INFORMATION_ID
```

Service ID of Ocotp_ReadApplicationInformation function

Definition at line 195 of file CDD_Ocotp.h.

6.1.3.23 OCOTP_WRITE_APPLICATION_INFORMATION_ID

```
#define OCOTP_WRITE_APPLICATION_INFORMATION_ID
```

Service ID of Ocotp_WriteApplicationInformation function

Definition at line 196 of file CDD_Ocotp.h.

6.1.3.24 OCOTP_ECC_ERROR_INJECT_ID

```
#define OCOTP_ECC_ERROR_INJECT_ID
```

Service ID of Ocotp_EccErrorInject function

Definition at line 197 of file CDD_Ocotp.h.

6.1.3.25 OCOTP_ECC_GET_ERROR_ID

```
#define OCOTP_ECC_GET_ERROR_ID
```

Service ID of Ocotp_EccGetError function

Definition at line 198 of file CDD_Ocotp.h.

6.1.3.26 OCOTP_SET_MODE_ID

```
#define OCOTP_SET_MODE_ID
```

Service ID of Ocotp_SetMode function

Definition at line 199 of file CDD_Ocotp.h.

6.1.3.27 OCOTP_CALCULATE_AND_WRITE_ECC_VALUE_ID

```
#define OCOTP_CALCULATE_AND_WRITE_ECC_VALUE_ID
```

Service ID of Ocotp_CalculateAndWriteCrcValue function

Definition at line 200 of file CDD_Ocotp.h.

6.1.3.28 OCOTP_CRC_VALIDATE_ID

```
#define OCOTP_CRC_VALIDATE_ID
```

Service ID of Ocotp_CrcValidate function

Definition at line 201 of file CDD_Ocotp.h.

6.1.3.29 OCOTP_READ_EFUSE_ID

```
#define OCOTP_READ_EFUSE_ID
```

Service ID of Ocotp_ReadEFuse function

Definition at line 202 of file CDD_Ocotp.h.

6.1.3.30 OCOTP_GET_STATUS_ID

```
#define OCOTP_GET_STATUS_ID
```

Service ID of Ocotp_GetStatus function

Definition at line 203 of file CDD_Ocotp.h.

6.1.4 Types Reference

6.1.4.1 Ocotp_AddressType

```
typedef uint32 Ocotp_AddressType
```

Ocotp address type.

Address of a register.

Definition at line 154 of file Ocotp_Types.h.

6.1.4.2 Ocotp_ChannelType

```
typedef uint8 Ocotp_ChannelType
```

Ocotp channel type.

An integer which is used to describe the order of channel configured.

Definition at line 161 of file Ocotp_Types.h.

6.1.5 Enum Reference

6.1.5.1 Ocotp_EccErrorType

```
enum Ocotp_EccErrorType
```

Ocotp ECC error type.

Enumerator

OCOTP_ECC_SINGLE_ERROR	ECC single error corect
OCOTP_ECC_DOUBLE_ERROR	ECC double error dectect
OCOTP_ECC_NO_ERROR	ECC no error occurred

Definition at line 120 of file Ocotp_Types.h.

6.1.5.2 Ocotp_ModeType

enum [Ocotp_ModeType](#)

Ocotp mode operation.

Enumerator

OCOTP_NORMAL_MODE	normal mode
OCOTP_POWERDOWN_MODE	power down mode

Definition at line 131 of file Ocotp_Types.h.

6.1.5.3 Ocotp_HardwareStatusType

enum [Ocotp_HardwareStatusType](#)

Status of Ocotp hardware.

Enumerator

OCOTP_HARDWARE_BUSY	Status of hardware is busy
OCOTP_HARDWARE_ERROR	Status of hardware is error
OCOTP_HARDWARE_IDLE	Status of hardware is idle

Definition at line 141 of file Ocotp_Types.h.

6.1.6 Function Reference

6.1.6.1 Ocotp_Init()

```
void Ocotp_Init (
    const Ocotp_ConfigType *const ConfigPtr )
```

The function initializes Ocotp module.

The function sets the internal module variables according to given configuration set.

Parameters

in	<i>ConfigPtr</i>	Pointer to Ocotp driver configuration set.
----	------------------	--

Precondition

Ocotp_pConfigPtr must be NULL_PTR.

6.1.6.2 Ocotp_Deinit()

```
void Ocotp_Deinit (
    void )
```

The function de-initializes Ocotp module.

The function sets the Ocotp module's status to un-initialized.

Parameters

in	<i>none.</i>	
----	--------------	--

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.1.6.3 Ocotp_WriteEFuse()

```
Std_ReturnType Ocotp_WriteEFuse (
    Ocotp_ChannelType eFuseChannel,
    uint32 data )
```

The function writes data to efuse word.

The function writes the specified data to the eFuse which was configured by configuration tool.

Parameters

in	<i>eFuseChannel</i>	Channel of eFuse such as 0, 1, 2 which was configured by user.
in	<i>data</i>	data user want to write to eFuse word.

Returns

Std_ReturnType

Return values

<i>E_OK</i>	Write operation is successful.
<i>E_NOT_OK</i>	Write operation failed.

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.1.6.4 Ocotp_WriteShadowRegister()

```
Std_ReturnType Ocotp_WriteShadowRegister (
    Ocotp_ChannelType eFuseChannel,
    uint32 data )
```

The function writes data to shadow register.

The function writes the specified data to the shadow register corresponding to eFuse which was configured by configuration tool.

Parameters

in	<i>eFuseChannel</i>	Channel of eFuse such as 0, 1, 2 which was configured by user.
in	<i>data</i>	data user want to write to shadow register.

Returns

Std_ReturnType

Return values

<i>E_OK</i>	Write operation is successful.
<i>E_NOT_OK</i>	Write operation failed.

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.1.6.5 Ocotp_ReadEFuse()

```
Std_ReturnType Ocotp_ReadEFuse (
    Ocotp_ChannelType eFuseChannel,
    uint32 * pData )
```

The function reads data from efuse word.

The function reads the data from the eFuse which was configured by configuration tool.

Parameters

in	<i>eFuseChannel</i>	Channel of eFuse such as 0, 1, 2 which was configured by user.
in	<i>data</i>	pointer points to the data read from eFuse word.

Returns

Std_ReturnType

Return values

<i>E_OK</i>	Read operation is successful.
<i>E_NOT_OK</i>	Read operation failed.

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.1.6.6 Ocotp_ReadShadowRegister()

```
Std_ReturnType Ocotp_ReadShadowRegister (
    Ocotp_ChannelType eFuseChannel,
    uint32 * pData )
```

The function writes data to shadow register.

The function writes the specified data to the shadow register corresponding to eFuse which was configured by configuration tool.

Parameters

in	<i>eFuseChannel</i>	Channel of eFuse such as 0, 1, 2 which was configured by user.
in	<i>data</i>	data user want to write to shadow register.

Returns

Std_ReturnType

Return values

<i>E_OK</i>	Write operation is successful.
<i>E_NOT_OK</i>	Write operation failed.

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.1.6.7 Ocotp_ReadApplicationInformation()

```
Std_ReturnType Ocotp_ReadApplicationInformation (
    Ocotp_ChannelType eFuseChannel,
    uint32 * pData )
```

The function reads data from GP register.

The function reads the data from the GP register which was configured in eFuse channel by configuration tool.

Parameters

in	<i>eFuseChannel</i>	Channel of eFuse such as 0, 1, 2 which was configured by user.
----	---------------------	--

Returns

Std_ReturnType

Return values

<i>E_OK</i>	read operation is successful.
<i>E_NOT_OK</i>	read operation failed.

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.1.6.8 Ocotp_WriteApplicationInformation()

```
Std_ReturnType Ocotp_WriteApplicationInformation (
    Ocotp_ChannelType eFuseChannel,
    uint32 data )
```

The function writes data to GP register.

The function writes the data to the GP register which was configured in eFuse channel by configuration tool.

Parameters

in	<i>eFuseChannel</i>	Channel of eFuse such as 0, 1, 2 which was configured by user.
in	<i>data</i>	data to be written.

Returns

Std_ReturnType

Return values

<i>E_OK</i>	write operation is successful.
<i>E_NOT_OK</i>	write operation failed.

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.1.6.9 Ocotp_EccErrorInject()

```
Std_ReturnType Ocotp_EccErrorInject (
    Ocotp_ChannelType eFuseChannel,
    uint8 bit )
```

The function injects ECC error in the specified eFuse.

The function inverts the value of the specified bit in the shadow register corresponding to eFuse which was configured in eFuse channel by configuration tool.

Parameters

in	<i>eFuseChannel</i>	Channel of eFuse such as 0, 1, 2 which was configured by user.
in	<i>bit</i>	bit's position which will be inverted.

Returns

Std_ReturnType

Return values

<i>E_OK</i>	inject operation is successful.
<i>E_NOT_OK</i>	inject operation failed.

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.1.6.10 Ocotp_EccGetError()

```
Std_ReturnType Ocotp_EccGetError (
    Ocotp_ChannelType eFuseChannel,
    Ocotp_EccErrorType * pEccValue )
```

The function gets ECC error of the specified eFuse.

The function check the single-bit and double-bit ECC error bit of the specified eFuse.

Parameters

in	<i>eFuseChannel</i>	Channel of eFuse such as 0, 1, 2 which was configured by user.
----	---------------------	--

Returns

Std_ReturnType

Return values

<i>E_OK</i>	get error operation is successful.
<i>E_NOT_OK</i>	get error operation failed.

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.1.6.11 Ocotp_SetMode()

```
Std_ReturnType Ocotp_SetMode (
    uint8 HwIndex,
    Ocotp_ModeType mode )
```

The function sets operation mode for Ocotp module.

The function sets operation mode for Ocotp module.

Parameters

in	<i>eFuseChannel</i>	Channel of eFuse such as 0, 1, 2 which was configured by user.
in	<i>mode</i>	mode which is to be applied (OCOTP_NORMAL_MODE, OCOTP_POWERDOWN_MODE).

Returns

Std_ReturnType

Return values

<i>E_OK</i>	mode is set successfully.
<i>E_NOT_OK</i>	set mode failed.

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.1.6.12 Ocotp_CalculateAndWriteCrcValue()

```
Std_ReturnType Ocotp_CalculateAndWriteCrcValue (
    Ocotp_ChannelType eFuseChannel,
    const uint32 * pData,
    uint32 dataSize )
```

The function writes the CRC value to the CRC fuse word.

The function calculates and writes the CRC value to the CRC fuse word which is configured in eFuse channel by configuration tool.

Parameters

in	<i>eFuseChannel</i>	Channel of eFuse such as 0, 1, 2 which was configured by user.
in	<i>pData</i>	Pointer points to the data buffer which need calculating CRC.
in	<i>dataSize</i>	Size of the data need calculating CRC in byte.

Returns

Std_ReturnType

Return values

<i>E_OK</i>	Write CRC value is successful.
<i>E_NOT_OK</i>	Write CRC value failed.

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.1.6.13 Ocotp_CrcValidate()

```
Std_ReturnType Ocotp_CrcValidate (
    Ocotp_ChannelType eFuseChannel )
```

The function validates the CRC.

The function compares the CRC value calculated by hardware to CRC value in the CRC fuse word which is configured in eFuse channel.

Parameters

in	<i>eFuseChannel</i>	Channel of eFuse such as 0, 1, 2 which was configured by user.
----	---------------------	--

Returns

Std_ReturnType

Return values

<i>E_OK</i>	Validate CRC value is successful.
<i>E_NOT_OK</i>	Validate CRC value failed.

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.1.6.14 Ocotp_GetStatus()

```
Std_ReturnType Ocotp_GetStatus (
    uint8 HwIndex,
    Ocotp_StatusType * pStatus )
```

The function gets status of Ocotp module.

The function gets status of Ocotp module Secure mode is enabled or not. Error occurred or not and Status of hardware.

Parameters

in	<i>HwIndex</i>	Index of hardware configured.
in	<i>pStatus</i>	Pointer points to status structure.

Returns

Std_ReturnType

Precondition

Ocotp_pConfigPtr must not be NULL_PTR.

6.2 OCOTP_IP

6.2.1 Detailed Description

Data Structures

- struct [Ocotp_Ip_ConfigType](#)
User configuration structure. [More...](#)

Types Reference

- typedef uint8 [Ocotp_InstanceType](#)
Ocotp instance type.

Enum Reference

- enum [Ocotp_Ip_EccErrorType](#)
ECC type (single or double error) will be checked.
- enum [Ocotp_Ip_SecureModeType](#)
FBXC secure mode.
- enum [Ocotp_Ip_ModeType](#)
FBXC mode operation.
- enum [Ocotp_Ip_StatusType](#)
Ocotp Ip status type.

Function Reference

- [Ocotp_Ip_StatusType](#) [Ocotp_Ip_Init](#) (const [Ocotp_Ip_ConfigType](#) *pConfig)
The function initializes Ocotp module.
- void [Ocotp_Ip_DeInit](#) (uint32 instance)
The function de-initializes Ocotp module.
- [Ocotp_Ip_StatusType](#) [Ocotp_Ip_WriteEFuse](#) (uint32 instance, uint32 address, uint32 data, boolean bEcc← Enable, boolean bBlocking)
The function writes data to the specified eFuse.
- [Ocotp_Ip_StatusType](#) [Ocotp_Ip_WriteShadowRegister](#) (uint32 instance, uint32 indexRegister, uint32 data)
The function writes data to the specified shadow register.
- [Ocotp_Ip_StatusType](#) [Ocotp_Ip_ReadEFuse](#) (uint32 instance, uint32 address, uint32 *data)
The function reads data from the specified eFuse.
- [Ocotp_Ip_StatusType](#) [Ocotp_Ip_ReadShadowRegister](#) (uint32 instance, uint32 indexRegister, uint32 *data)
The function reads data from the specified shadow register.
- boolean [Ocotp_Ip_EccGetError](#) (uint32 instance, uint32 address, [Ocotp_Ip_EccErrorType](#) typeECC)
The function gets ECC error from the specified eFuse.
- void [Ocotp_Ip_ClearErrorFlags](#) (uint32 instance)

The function clears all errors flags.

- uint32 [Ocotp_Ip_ReadGpRegister](#) (uint32 instance, uint32 indexRegister)

The function reads data from the specified GP register.

- void [Ocotp_Ip_WriteGpRegister](#) (uint32 instance, uint32 indexRegister, uint32 data)

The function writes data to the specified GP register.

- void [Ocotp_Ip_EccErrorInject](#) (uint32 instance, uint32 indexRegister, uint32 bit)

The function injects Ecc error into the specified shadow register.

- [Ocotp_Ip_StatusType](#) [Ocotp_Ip_Setmode](#) (uint32 instance, [Ocotp_Ip_ModeType](#) mode)

The function sets mode operation for Ocotp.

- [Ocotp_Ip_StatusType](#) [Ocotp_Ip_GetBusyState](#) (uint32 instance)

The function checks busy and error state of hardware.

- [Ocotp_Ip_StatusType](#) [Ocotp_Ip_CalculateAndWriteCrcValue](#) (uint32 instance, uint32 CRCEfuseAddress, const uint32 *dataPtr, uint32 dataSize)

The function calculates and writes CRC value to CRC fuse word base on the input data.

- [Ocotp_Ip_StatusType](#) [Ocotp_Ip_CrcTest](#) (uint32 instance, uint32 crcEFuseWordAddress, uint32 startAddress, uint32 endAddress)

tests CRC32 in specified region

6.2.2 Data Structure Documentation

6.2.2.1 struct Ocotp_Ip_ConfigType

User configuration structure.

Structure contains the configuration parameters which will be used to initialize Ocotp module.

Definition at line 134 of file Ocotp_Ip_Types.h.

Data Fields

Type	Name	Description
Ocotp_InstanceType	ocotpInstance	Ocotp instance
Ocotp_Ip_SecureModeType	secureMode	Secure mode

6.2.3 Types Reference

6.2.3.1 Ocotp_InstanceType

```
typedef uint8 Ocotp\_InstanceType
```

Ocotp instance type.

An integer which describe the order of the instance configured.

Definition at line 128 of file Ocotp_Ip_Types.h.

6.2.4 Enum Reference

6.2.4.1 Ocotp_Ip_EccErrorType

enum `Ocotp_Ip_EccErrorType`

ECC type (single or double error) will be checked.

Enumerator

OCOTP_IP_ECC_SINGLE_ERROR	ECC single error check
OCOTP_IP_ECC_DOUBLE_ERROR	ECC double error check

Definition at line 84 of file `Ocotp_Ip_Types.h`.

6.2.4.2 Ocotp_Ip_SecureModeType

enum `Ocotp_Ip_SecureModeType`

FBXC secure mode.

Enumerator

OCOTP_FBXC_NONE_SECURE_MODE	None secure mode mode
OCOTP_FBXC_SECURE_MODE	Secure mode mode

Definition at line 94 of file `Ocotp_Ip_Types.h`.

6.2.4.3 Ocotp_Ip_ModeType

enum `Ocotp_Ip_ModeType`

FBXC mode operation.

Enumerator

OCOTP_FBXC_NORMAL_MODE	normal run mode
OCOTP_FBXC_POWER_DOWN_MODE	Powerdown mode

Definition at line 103 of file `Ocotp_Ip_Types.h`.

6.2.4.4 Ocotp_Ip_StatusType

enum `Ocotp_Ip_StatusType`

Ocotp Ip status type.

Enumerator

<code>STATUS_OCOTP_IP_ERROR</code>	Status error
<code>STATUS_OCOTP_IP_BUSY</code>	Status busy
<code>STATUS_OCOTP_IP_SUCCESS</code>	Status success
<code>STATUS_OCOTP_IP_INVALID_INPUT_ADDRESS</code>	invalid input parameter
<code>STATUS_OCOTP_IP_ERROR_TIMEOUT</code>	timeout error

Definition at line 112 of file `Ocotp_Ip_Types.h`.

6.2.5 Function Reference

6.2.5.1 Ocotp_Ip_Init()

```
Ocotp_Ip_StatusType Ocotp_Ip_Init (
    const Ocotp_Ip_ConfigType * pConfig )
```

The function initializes Ocotp module.

The function clear all errors flag and initializes Ocotp module.

Parameters

in	<i>pConfig</i>	pointer points to configuration structure.
----	----------------	--

Returns

`Ocotp_Ip_StatusType`

Return values

<code>STATUS_OCOTP_IP_SUCCESS</code>	if module is initialized successfully.
--------------------------------------	--

Precondition

Module has been initialized.

6.2.5.2 Ocotp_Ip_DeInit()

```
void Ocotp_Ip_DeInit (
    uint32 instance )
```

The function de-initializes Ocotp module.

The function de-initializes Ocotp module.

Parameters

in	<i>instance</i>	hardware instance.
----	-----------------	--------------------

Precondition

Module has been initialized.

6.2.5.3 Ocotp_Ip_WriteEFuse()

```
Ocotp_Ip_StatusType Ocotp_Ip_WriteEFuse (
    uint32 instance,
    uint32 address,
    uint32 data,
    boolean bEccEnable,
    boolean bBlocking )
```

The function writes data to the specified eFuse.

The function writes data to the specified eFuse.

Parameters

in	<i>instance</i>	hardware instance.
in	<i>address</i>	address of eFuse.
in	<i>data</i>	data to be written.
in	<i>eccEnable</i>	enable ECC protection for current fuse word or not.
in	<i>blocking</i>	TRUE: write operation is blocking. FALSE: otherwise.

Module Documentation

Returns

Ocotp_Ip_StatusType

Return values

<i>STATUS_OCOTP_IP_SUCCESS</i>	if write operation is successful.
<i>STATUS_OCOTP_IP_ERROR</i>	if error flag is set.
<i>STATUS_OCOTP_IP_ERROR_TIMEOUT</i>	timeout supervision is enabled and timeout occurred.
<i>STATUS_OCOTP_IP_BUSY</i>	hardware is busy.

Precondition

Module has been initialized.

6.2.5.4 Ocotp_Ip_WriteShadowRegister()

```
Ocotp_Ip_StatusType Ocotp_Ip_WriteShadowRegister (
    uint32 instance,
    uint32 indexRegister,
    uint32 data )
```

The function writes data to the specified shadow register.

The function writes data to the specified shadow register.

Parameters

in	<i>instance</i>	hardware instance.
in	<i>indexRegister</i>	index of shadow register.
in	<i>data</i>	data to be written.

Returns

Ocotp_Ip_StatusType

Return values

<i>STATUS_OCOTP_IP_SUCCESS</i>	if write operation is successful.
<i>STATUS_OCOTP_IP_ERROR</i>	if error flag is set.
<i>STATUS_OCOTP_IP_BUSY</i>	hardware is busy.

Precondition

Module has been initialized.

6.2.5.5 Ocotp_Ip_ReadEFuse()

```
Ocotp_Ip_StatusType Ocotp_Ip_ReadEFuse (
    uint32 instance,
    uint32 address,
    uint32 * data )
```

The function reads data from the specified eFuse.

The function reads data from the specified eFuse.

Parameters

in	<i>instance</i>	hardware instance.
in	<i>address</i>	address of eFuse.
in	<i>data</i>	pointer points to data result.

Returns

Ocotp_Ip_StatusType

Return values

<i>STATUS_OCOTP_IP_SUCCESS</i>	if read operation is successful.
<i>STATUS_OCOTP_IP_ERROR</i>	if error flag is set.
<i>STATUS_OCOTP_IP_ERROR_TIMEOUT</i>	timeout supervision is enabled and timeout occurred.
<i>STATUS_OCOTP_IP_BUSY</i>	hardware is busy.

Precondition

Module has been initialized.

6.2.5.6 Ocotp_Ip_ReadShadowRegister()

```
Ocotp_Ip_StatusType Ocotp_Ip_ReadShadowRegister (
    uint32 instance,
```

Module Documentation

```
uint32 indexRegister,  
uint32 * data )
```

The function reads data from the specified shadow register.

The function reads data from the specified shadow register.

Parameters

in	<i>instance</i>	hardware instance.
in	<i>indexRegister</i>	index of shadow register.
in	<i>data</i>	pointer points to data result.

Returns

Ocotp_Ip_StatusType

Return values

<i>STATUS_OCOTP_IP_SUCCESS</i>	if read operation is successful.
<i>STATUS_OCOTP_IP_ERROR</i>	if error flag is set.
<i>STATUS_OCOTP_IP_BUSY</i>	hardware is busy.

Precondition

Module has been initialized.

6.2.5.7 Ocotp_Ip_EccGetError()

```
boolean Ocotp_Ip_EccGetError (
    uint32 instance,
    uint32 address,
    Ocotp_Ip_EccErrorType typeECC )
```

The function gets ECC error from the specified eFuse.

The function gets ECC error from the specified eFuse.

Parameters

in	<i>instance</i>	hardware instance.
in	<i>address</i>	address of eFuse.
in	<i>typeECC</i>	type of ECC to be checked(OCOTP_IP_ECC_SINGLE_ERROR, OCOTP_IP_ECC_DOUBLE_ERROR).

Returns

boolean

Return values

<i>TRUE</i>	if ECC error flag is set.
<i>FALSE</i>	if ECC error flag is not set.

Precondition

Module has been initialized.

6.2.5.8 Ocotp_Ip_ClearErrorFlags()

```
void Ocotp_Ip_ClearErrorFlags (  
    uint32 instance )
```

The function clears all errors flags.

The function clears all errors flags.

Parameters

in	<i>instance</i>	hardware instance.
----	-----------------	--------------------

Precondition

Module has been initialized.

6.2.5.9 Ocotp_Ip_ReadGpRegister()

```
uint32 Ocotp_Ip_ReadGpRegister (  
    uint32 instance,  
    uint32 indexRegister )
```

The function reads data from the specified GP register.

The function reads data from the specified GP register.

Parameters

in	<i>instance</i>	hardware instance.
in	<i>indexRegister</i>	index of GP register.

Returns

uint32

Return values

<i>data</i>	read from GP register
-------------	-----------------------

Precondition

Module has been initialized.

6.2.5.10 Ocotp_Ip_WriteGpRegister()

```
void Ocotp_Ip_WriteGpRegister (
    uint32 instance,
    uint32 indexRegister,
    uint32 data )
```

The function writes data to the specified GP register.

The function writes data to the specified GP register.

Parameters

in	<i>instance</i>	hardware instance.
in	<i>indexRegister</i>	index of GP register.
in	<i>data</i>	data to be written.

Precondition

Module has been initialized.

6.2.5.11 Ocotp_Ip_EccErrorInject()

```
void Ocotp_Ip_EccErrorInject (
    uint32 instance,
    uint32 indexRegister,
    uint32 bit )
```

The function injects Ecc error into the specified shadow register.

The function inverts value in the specified bit postion in shadow register.

Module Documentation

Parameters

in	<i>instance</i>	hardware instance.
in	<i>indexRegister</i>	index of shadow register.
in	<i>bit</i>	bit position which will be inverted.

Precondition

Module has been initialized.

6.2.5.12 Ocotp_Ip_Setmode()

```
Ocotp_Ip_StatusType Ocotp_Ip_Setmode (  
    uint32 instance,  
    Ocotp_Ip_ModeType mode )
```

The function sets mode operation for Ocotp.

The function sets mode operation for Ocotp. If mode is OCOTP_FBXC_NORMAL_MODE: module is in normal running mode. If mode is OCOTP_FBXC_POWER_DOWN_MODE: module is in powerdown mode.

Parameters

in	<i>instance</i>	hardware instance.
in	<i>mode</i>	mode to be set.

Returns

Ocotp_Ip_StatusType

Return values

<i>STATUS_OCOTP_IP_SUCCESS</i>	if set mode operation is successful.
<i>STATUS_OCOTP_IP_ERROR</i>	if error flag is set.
<i>STATUS_OCOTP_IP_BUSY</i>	hardware is busy.
<i>STATUS_OCOTP_IP_ERROR_TIMEOUT</i>	timeout supervision is enabled and timeout occurred.

Precondition

Module has been initialized.

6.2.5.13 Ocotp_Ip_GetBusyState()

```
Ocotp_Ip_StatusType Ocotp_Ip_GetBusyState (
    uint32 instance )
```

The function checks busy and error state of hardware.

The function checks busy and error state of hardware.

Parameters

in	<i>instance</i>	hardware instance.
----	-----------------	--------------------

Returns

Ocotp_Ip_StatusType

Return values

<i>STATUS_OCOTP_IP_SUCCESS</i>	if hardware is ready.
<i>STATUS_OCOTP_IP_ERROR</i>	if error flag is set.
<i>STATUS_OCOTP_IP_BUSY</i>	hardware is busy.

Precondition

Module has been initialized.

6.2.5.14 Ocotp_Ip_CalculateAndWriteCrcValue()

```
Ocotp_Ip_StatusType Ocotp_Ip_CalculateAndWriteCrcValue (
    uint32 instance,
    uint32 CRCEfuseAddress,
    const uint32 * dataPtr,
    uint32 dataSize )
```

The function calculates and writes CRC value to CRC fuse word base on the input data.

The function calculates and writes CRC value to CRC fuse word base on the input data.

Parameters

in	<i>instance</i>	hardware instance.
in	<i>CRCEfuseAddress</i>	address of CRC fuse word.
in	<i>dataPtr</i>	input data pointer.
in	<i>dataSize</i>	size of data(in 4 bytes).

Module Documentation

Returns

Ocotp_Ip_StatusType

Return values

<i>STATUS_OCOTP_IP_SUCCESS</i>	if write CRC value is successful.
<i>STATUS_OCOTP_IP_ERROR</i>	if error flag is set.
<i>STATUS_OCOTP_IP_BUSY</i>	hardware is busy.

Precondition

Module has been initialized.

6.2.5.15 Ocotp_Ip_CrcTest()

```
Ocotp_Ip_StatusType Ocotp_Ip_CrcTest (
    uint32 instance,
    uint32 crcEFuseWordAddress,
    uint32 startAddress,
    uint32 endAddress )
```

tests CRC32 in specified region

Parameters

<i>instance</i>	OCOTP peripheral instance number
<i>crcEFuseWordAddress</i>	address of CRC fuse word
<i>startAddress</i>	start address of the specified region
<i>endAddress</i>	end address of the specified region

Returns

STATUS_OCOTP_IP_SUCCESS - CRC test passed. or otherwise CRC test failed

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, Altivec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2023 NXP B.V.

